

Ella Potter & James Whipple

Professor Beichel

ECE:3360 Embedded Systems

Post-Lab Report 5

1. Introduction

The main goal of this lab was to build a remotely controllable analog data logging system. This system should be able to output voltages between 0 and 5 V using a digital-to-analog converter and read voltages between 0 and 5 V using an analog-to-digital converter. The ADC used will be the one built into the ATmega328P, while the DAC used will be an external component, the MAX518. The other part of this lab was implementing an RS232 interface to be able to communicate with a user through the laptop. The user should be able to insert 'G' to get one voltage measurement from the ADC. To get multiple voltage measurements, they can enter 'M,n,dt' where n is the number of measurements and dt is the time between measurements. Finally, the user should be able to enter 'S,c,v' to set the DAC output voltage, where c is the channel number (0 or 1) and v is the voltage value to be set.

2. Schematic

Figure 1 shows the circuit set-up for this Lab, using only the Arduino and the MAX518. The ADC is implemented inside the Arduino, so that is all set up through code. The DAC on the other hand, is the one component we did have to wire to the Arduino. Please note that A0 is used to read the ADC input, so it could be wired to OUT0, OUT1, or any other point on the breadboard.

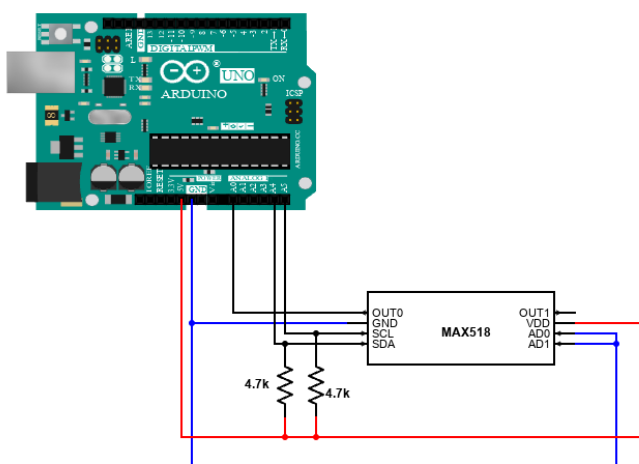


Figure 1: Circuit Diagram

3. Discussion

3.1 Hardware

The hardware for this lab contained only one new component since the ADC is built into our ATmega328P. Therefore, the only circuitry needed was for the MAX518.

Pin on Max518	Function	Connection to ATmega328P
OUT0	DAC output voltage	PC0 (A0)
GND	Ground Connection	GND
SCL	Serial Clock Input	PC5 (A5)
SDA	Serial Data Input	PC4 (A4)
OUT1	DAC output voltage	PC0 (A0)
VDD	Power Supply	5V
AD0	Address Input 0	GND
AD1	Address Input 1	GND

- OUT0 and OUT1 are both outputs for the DAC and will output an analog signal representing the specific voltage that the user has set it to, if it has been set.
 - If the user chooses to set channel 0 the voltage will show up on OUT0 where as if they choose to set channel 1 the voltage will show up on OUT1.
 - These voltages can then be read using the ADC by connecting it to A0, which is ADC0 on the Arduino.
- GND and VDD are just simple ground and voltage connections that are used to help convert the digital signal to an analog signal.
- AD0 and AD1 define the two-bit slave address for the device. These could be set at either 0 or 1 values.
 - For simplicity, we grounded both of them making the address just 00.
- Finally, SCL and SDA control the data sending the receiving of the DAC.
 - SCL will be used as the systems clock signal, where SDA will be used as the system's data signal to receive digital data from the Arduino.
 - These are wired to the SCL and SDA pins on the Arduino, which are A5 and A4.

3.2 Software

After setting up the circuit, we wrote the logic for the system using C language. We will break down the software for the different components of the system.

3.2.1 RS232

We used the RS232 on the Arduino for serial communication of characters. This allowed us to be able to both transmit/send data to an output screen as well as

receive/read the This user's input. To implement this, we used the microcontroller USART0 (Universal Synchronous-Asynchronous Receiver-Transmitter).

- **USART_Init:** This function is called at the beginning of execution and initializes the USART so it is ready to be used. To do this, it sets the baud rate, enables the receiver and transmitter, and sets the frame format.
 - Baud Rate = 9600
 - Frame Format: 10-bit mode, 8 data bits, 2 stop bits
- **USART_Transmit:** This function will send one character to the output screen for the user's viewing. To do this, it must wait for an empty transmit buffer by continuously looping, and then, when one is open, it will place this data into the buffer to be sent.
 - Only used to send one character, use USART_Print to send multiple
- **USART_Receive:** This function will return one character from user input. To do this, it waits for the buffer to have data and will then return that data.
- **USART_Print:** This function is an extension of USART_Transmit and will send a whole string of characters to the output screen. To do this it repeatedly calls USART_Transmit for each character in the screen.

***Note:** The above functions were found in the ATmega328P datasheet underneath the USART0 section. The only function not found there is the USART_Print.

3.2.2 ADC

After setting up the serial communication, we were ready to start reading voltages and communicating them to the user. This required the use of the ATmega328P's built-in ADC, which would be able to read an analog voltage signal from the breadboard and convert it to a digital voltage signal to be shown to the user.

- **ADC_Init:** This function is called at the beginning of execution and will initialize the ADC to be ready for use. To do this, it must set the ADMUX and ADCSRA registers.
 - **ADMUX :** This register controls the multiplexer in the ADC.
 - Bits 7 – 6 (REFS1, REFS0) : these select the reference voltage that will be used in the ADC comparison
 - Bits 3 – 0 (MUX3 – MUX0): these select the analog input to be input into the ADC
 - Should be set as 01000000, which will set Vcc as our reference voltage and PC0 as our analog input
 - **ADCSRA:** This register is the control and status register for the ADC
 - Bit 7 (ADEN): this bit enables/disables the ADC. If set to 1, the ADC will be turned on; if set to 0, the ADC will turn off.

- Bit 6 (ADSC): this bit will start the ADC conversion, given that the ADC is enabled.
- Bits 2 – 0 (ADPS): these bits set the ADC prescaler
- Should be set as 10000111, which will enable the ADC and set the prescaler as 128
- **getVoltage:** This function is called anytime a user inputs ‘G’ and will use the ADC to read an analog voltage from the breadboard and output it as a digital voltage to the screen.
 - To begin the ADC conversion, we must set the ADSC bit in the ADCSRA register to 1 (ADCSRA = 11000111)
 - After this new bit is loaded, we will then read the ADC value from PC0 and store it in a float ADC
 - To determine the voltage value we can use the following equation: $V_{in} = (V_{ref} \times ADC) \div 1024$; where $V_{ref} = 5V$
 - Once V_{in} is calculated, we simply use `USART_Print` to display the results to the user.

3.2.2 DAC

Now that the ADC had been set up, it was time to set up the final component of the system, the DAC. We used Peter Fleury’s I2C Master Library to help with some of the code for communicating and setting up the MAX518. The following will first describe the registers used in a two-wire serial interface and then describe the functions used from Fleury’s “twimaster.c” class, which are derived from `i2cmaster.h`.

TWBR : TWI Bit Rate Register		
Bits 7 – 0	TWBR7 – TWBR0	Sets the division factor used in calculating the SCL frequency.

TWCR: TWI Control Register		
Bit 7	TWINT	Interrupt flag bit, used to indicate that the TWI is finished with its current job.
Bit 6	TWEA	Enable acknowledge bit, used to generate an acknowledgement pulse.
Bit 5	TWSTA	Start condition bit, used to indicate that the application wants to become a master to the TWI bus and will generate a start condition if the bus is free.
Bit 4	TWSTO	Stop condition bit, used to generate a stop condition on the TWI bus.
Bit 3	TWWC	Write collision flag, used to indicate when the application is attempting to write to the TWI data register.
Bit 2	TWEN	Enable bit, used to enable TWI operation and activate the TWI interface.

Bit 1	TWIE	Interrupt Enable, used to enable interrupts for the TWI.
-------	------	--

TWSR: TWI Status Register		
Bits 7 - 3	TWS7 – TWS3	These bits represent the status of the TWI logic and bus interface. The specific codes can be found in the datasheet.
Bits 1 - 0	TWPS1 – TWPS0	These bits select the prescalar value used in the SCL frequency equation.

TWDR: TWI Data Register		
Bits 7 - 0	TWD7 – TWD0	This register contains the next byte to be transmitted.

$$SCL \text{ frequency} = CPU \text{ frequency} \div (16 + 2 (TWBR) * Prescalar)$$

- **i2c_init:** This function is used to initialize the 2-wire serial interface to be able to communicate with the MAX518. To do this it must set the TWSR and TWBR registers to set up the TWI clock.
 - **TWSR:** For initialization, we only need to set the prescalar value. We will use a prescalar of 1, so TWSR = 00000000.
 - **TWBR:** We also need to set the division factor, which can be calculated using the SCL frequency equation above.
- **i2c_start:** This function is used to send a start condition to the TWI as well as send the slave byte containing the address.
 - We first must send the start condition by setting TWINT, TWSTA, and TWEN all at 1 in the TWI control register. We will then loop until all these bits are latched.
 - Next, we will check the value of the status register and make sure that the start condition has been received
 - We can now send the slave byte by loading it into the TWI data register and then setting TWINT and TWEN as 1 in the control register. Again, we should loop until these bits are latched.
 - Finally, we will check the value of the status register and make sure that our byte has been acknowledged
- **i2c_write:** This function will send one byte of data to the I2C device.
 - First, it will load the byte into the data register and then set TWINT and TWEN as 1 in the control register. It will then wait until these bits are latched before moving on.
 - Next, it will check the value of the status register and confirm that the byte was acknowledged.
- **i2c_stop:** This function is used to send the stop condition to the TWI, which will stop the data transfer and release the bus. To do this it just sets TWINT, TWEN,

and TWSTO as 1 in the control register and waits until the stop condition is executed.

Extra Functions

Now, that all 3 components are set up, the rest of the coding was easy. We just needed to implement functions to read the user inputs for the M and S functions as well as implement functions for getting multiple voltages and setting out DAC output.

- **Main:** The main function will first initialize the USART, the ADC, and the DAC. It will then prompt the user with the different commands they can enter and what the commands do. Finally, it will continuously loop and check for user input.
 - If the user inputs a G, it will call `getVoltage`, which will display one voltage measurement
 - If the user inputs an M or an S, it will call `readInput_M` or `readInput_S` which will read in the following inputs after the letter
 - Otherwise, if the user inputs an invalid command, it will output that back to the user
- **readInput_M:** This function will read in the n and dt input from the user, if they were given. It uses `USART_Receive` to receive one character at a time and checks that they are valid inputs. If all characters are valid n and dt are transformed into ints and then passed to `getMultiple(n,dt)`.
- **getMultiple:** This function loops n times and calls `getVoltage` each time followed by a dt-second delay.
- **readInput_S:** This function is similar to `readInput_M` as it makes sure all characters are given and valid, except it has to take in an int and a float. If all characters are valid and c and v are in their required range, then they will be passed to `setDAC(c,v)`.
- **setDAC:** This function is used to communicate with the DAC and set the output voltage on a certain channel determined by the user's input.
 - First, it will call `i2c_start` and pass in the slave address to start transmission and set the address of our DAC as 00 since we grounded AD0 and AD1
 - Next, we send the command byte using `i2c_write`, which will set the channel that the DAC voltage will be sent to.
 - Next, we will send the actual DAC value using `i2c_write`. This value can be calculated by the following: $DAC = (V_{in} * 255) / 5$
 - Finally, we will call `i2c_stop` to stop the transmission.

4. Conclusion

From this lab, we have gained a lot of knowledge in C, serial communication, and analog-to-digital conversions. Since this was our first lab writing in C we had to learn the differences between C and assembly language, which did not take long to get the hang of and ended up being a way easier and faster language to write in. We also learned a lot about serial communication, both to our computer and to other hardware components. This will be helpful as we go into the final project and have to use this knowledge on our own I2C device. Lastly, we learned a lot about the ATmega's built-in ADC and how to initialize and run it. This lab has given us the knowledge we need to go into our final project.

5. Appendix A: Source Code

```

6. #define F_CPU 16000000UL
7. #define BAUD 9600
8. #define MYUBRR F_CPU/16/BAUD-1
9.
10.
11. #include <avr/io.h>
12. #include <math.h>
13. #include <string.h>
14. #include <util/delay.h>
15. #include <stdlib.h>
16. #include <stdio.h>
17. #include "i2cmaster.h"
18.
19.
20. //USART Functions
21. void USART_Init(unsigned int ubrr){ //Initializes USART
22.     //Set baud rate
23.     UBRR0H = (unsigned char) (ubrr>>8);
24.     UBRR0L = (unsigned char) ubrr;
25.     //Enable Rx and Tx
26.     UCSR0B = (1<<RXEN0)|(1<<TXEN0);
27.     //Set frame format
28.     UCSR0C = (1<<USBS0) | (3<<UCSZ00);
29. }
30.
31. void USART_Transmit(unsigned char data){ //sends one char at a time to output
    screen
32.     //Wait for empty buffer
33.     while(!(UCSR0A & (1<<UDRE0)));
34.     //Put data into buffer, sends data
35.     UDR0 = data;
36. }
37.
38. unsigned char USART_Recieve(void){ //receives on char at a time from the user
    input
39.     //Wait for data to be recieved
40.     while(!(UCSR0A & (1<<RXC0)));
41.     //Get and return data from buffer
42.     return UDR0;
43. }

```

```

44.
45. void USART_Print(char string[]){    //sends a string of characters to output
    screen
46.    //send each character bit by bit to screen
47.    for(int i = 0; i < strlen(string); i++){
48.        USART_Transmit(string[i]);
49.    }
50. }
51.
52.
53. //ADC Functions
54. void ADC_Init(void){ //Initializes the ADC
55.     ADMUX = (1<<REFS0); //Set voltage supply as Vcc and set ADC pin to PC0
56.     ADCSRA = (1<<ADEN)| (1<<ADPS2)| (1<<ADPS1)| (1<<ADPS0); //Enable ADC and
    Set prescalar to 128
57. }
58.
59.
60. void getVoltage(void){ //receives the voltage input from pin PC0
61.     float adc; //used to store value of ADC
62.     char sVolt[50]; //stores the voltage value in string format to be
    printed
63.
64.     ADCSRA = (1<<ADEN)|(1<<ADSC)|(1<<ADPS2)| (1<<ADPS1)| (1<<ADPS0); //set
    register to start ADC conversion
65.
66.     while(ADCSRA & (1<<ADSC)); //wait for new value of ADSC to latch
67.
68.     adc = ADC; //read in ADC value
69.
70.     adc = (5*adc)/1024.0; //convert to voltage
71.
72.     dtostrf(adc, 5, 3, sVolt); //convert adc into a string value and store in
    sVolt
73.
74.     USART_Print("Voltage = "); //prints out the voltage reading to the user
75.     USART_Print(sVolt);
76.     USART_Print(" V\n");
77. }
78.
79. void readInput_M(void){
80.     unsigned char comma1 = USART_Recieve(); //reads in the first comma after M
81.     int n;
82.     int dt;
83.     int valid = 1; //keeps track of if the input is valid, if it goes to -1
    input is invalid
84.     if(comma1 == ','){ //if the first input read in was a comma then we can
    start reading n
85.         unsigned char n1 = USART_Recieve(); //receive first value of n
86.         unsigned char maybe_n2 = USART_Recieve(); //receive either second
    value of n or a comma if n is one digit
87.         if(maybe_n2 == ','){ //if n2 is a comma then n is only one digit
88.             char nString[2];
89.             nString[0] = n1;
90.             nString[1] = '\0';
91.             n = atoi(nString); //therefore we can turn n1 into an int and
    store in n

```



```

92.         }else{           //otherwise if n2 isnt a comma then n is more then one
digit
93.             char nString[3]; //we can store both digits in a string
94.             nString[0] = n1;
95.             nString[1] = maybe_n2;
96.             nString[2] = '\0';
97.             n = atoi(nString); //change the string to an int and store in
n
98.             unsigned char maybe_comma2 = USART_Recieve(); //read in the
next value after the 2 n digits
99.             if(maybe_comma2 != ','){ //if the value is not a comma then
the user tried to input a 3 digit n value
100.                 USART_Print("Invalid input for n\n");
101.                 valid = -1; //change to invalid input as n is
too many digits
102.             }
103.         }
104.         if(n < 2 || n > 20){ //finally make sure that the n value is
between 2 and 20
105.             USART_Print("Invalid input for n: must be between 2 and
20\n");
106.             valid = -1; //if its not change to invalid
107.         }
108.     }else{ //if M is not followed by a comma change to invalid
109.         USART_Print("M must be followed by a comma\n");
110.         valid = -1;
111.     }
112.
113.     if(valid == 1){ //make sure that input is still valid
114.         unsigned char dt1 = USART_Recieve(); //take in first digit of
dt
115.         unsigned char maybe_dt2 = USART_Recieve(); //take in second
digit of dt or send of string
116.         if(maybe_dt2 == '\0' || maybe_dt2 == '\n'){ //if the second
input is the end of the string dt is only one digit
117.             char dtString[2];
118.             dtString[0] = dt1;
119.             dtString[1] = '\0';
120.             dt = atoi(dtString); // we can transform dt1 to an int
and store in dt
121.         }else{ //otherwise dt is more then one digit
122.             char dtString[3];
123.             dtString[0] = dt1;
124.             dtString[1] = maybe_dt2;
125.             dtString[2] = '\0';
126.             dt = atoi(dtString); //change both digits of dt into
an int value and store
127.             unsigned char maybe_end = USART_Recieve(); //take in
the final value and make sure its the end
128.             if(maybe_end != '\0' && maybe_end != '\n'){
129.                 USART_Print("Invalid input for dt\n");
130.                 valid = -1; //if the final value is not the end
of the string then dt is too many digits
131.             }
132.         }
133.         if(dt < 1 || dt > 10){ //make sure dt is between 1 and 10
134.             USART_Print("Invalid input for dt: must be between 1
and 10\n");

```

```

135.         valid = -1;
136.     }
137. }
138.     if(valid == 1){ //finally make sure the input is still valid
139.         getMultiple(n, dt); //if it is call getMultiple with the n and
dt values from user
140.     }
141. }
142.
143. void getMultiple(int n, int dt){
144.     for(int i = 0; i < n; i++){ //repeats n times where n is from user
input
145.         int currentTime = i*dt; //compute the current time passed in
the loop
146.         char timeString[50];
147.         itoa(currentTime, timeString, 10);
148.         USART_Print("Time: ");
149.         USART_Print(timeString);
150.         USART_Print(" s\n"); //output the current time to the user
151.         getVoltage(); // get the ADC voltage reading
152.         for(int j = 0; j < dt; j++){ //delay for dt seconds
153.             _delay_ms(1000);
154.         }
155.     }
156. }
157.
158. void readInput_S(void){
159.     unsigned char s_comma1 = USART_Recieve(); //reads in the first comma
after S
160.     int s_valid = 1; //keeps track of if valid input (-1 : invalid)
161.     int channelNum; //holds channel input given
162.     float v; //holds v input given
163.     if(s_comma1 == ','){ //make sure comma after S
164.         unsigned char channel = USART_Recieve(); //read in channel
number
165.         unsigned char s_comma2 = USART_Recieve(); //read in next
input, should be a comma
166.         if(s_comma2 == ','){ //if its a comma then its still valid
167.             char cString[2]; //store in string and changes to int
168.             cString[0] = channel;
169.             cString[1] = '\0';
170.             channelNum = atoi(cString);
171.             if(channelNum > 1 || channelNum < 0){ //make sure
channel input is 0 or 1
172.                 USART_Print("Invalid input for c: must be 0 or
1\n");
173.                 s_valid = -1;
174.             }
175.         }else{ //if s_comma2 was not a comma then invalid input
176.             USART_Print("C must be followed by a comma\n");
177.             s_valid = -1;
178.         }
179.     }else{ //if s_comma1 was not a comma then invalid input
180.         USART_Print("S must be followed by a comma\n");
181.         s_valid = -1;
182.     }
183.     if(s_valid == 1){ //make sure the input is still valid before
recieving v

```

```

184.         unsigned char float1 = USART_Recieve(); //receive first digit
of v
185.         unsigned char dot = USART_Recieve(); //receive the decimal
point
186.         if(dot == '.'){ //make sure that the user gave a float
187.             unsigned char float2 = USART_Recieve(); //receive the
last two digits
188.             unsigned char float3 = USART_Recieve();
189.             if(float2 == '\0' || float2 == '\n'){ //make sure user
gave digit after decimal
190.                 USART_Print("V must be a float: include 2
decimal places\n");
191.                 s_valid = -1;
192.             }
193.             if(float3 == '\0' || float3 == '\n'){ //and the second
digit after decimal
194.                 USART_Print("V must be a float: include 2
decimal places\n");
195.                 s_valid = -1;
196.             }
197.             if(s_valid == 1){ //make sure that the float input was
valid before moving on
198.                 char floatString[5];
199.                 floatString[0] = float1;
200.                 floatString[1] = dot;
201.                 floatString[2] = float2;
202.                 floatString[3] = float3;
203.                 floatString[4] = '\0';
204.                 v = atof(floatString); //change into a float and
store in v
205.                 if(v > 5 || v < 0){ //make sure that v is
between 0 and 5
206.                     USART_Print("Invalid input for v: must be
between 0 and 5");
207.                     s_valid = -1;
208.                 }else{ //finally, if v is valid call setDAC
209.                     setDAC(channelNum,v);
210.                 }
211.             }
212.         }else{ //if decimal point not included
213.             USART_Print("V must be a float: include decimal
214.             point\n");
215.             s_valid = -1;
216.         }
217.     }
218. }
219.
220.
221.
222. void setDAC(int c_in, float v_in){
223.     char vString[50];
224.     dtostrf(v_in, 4, 2, vString);
225.     char dac_char[50];
226.     int dac = (v_in * 255 ) / 5; //compute DAC value to send
227.     itoa(dac, dac_char, 10);
228.
229.     //Print results to user

```

```

230.         if(c_in == 1){
231.             USART_Print("DAC channel 1 set to ");
232.         }else{
233.             USART_Print("DAC channel 0 set to ");
234.         }
235.         USART_Print(vString);
236.         USART_Print(" V (");
237.         USART_Print(dac_char);
238.         USART_Print("d)\n");
239.
240.
241.         unsigned char command; //stores the command data to be sent to DAC
242.         if(c_in == 1){
243.             command = 0b00000001; //if user selected channel 1 set OUT1 as
output
244.             }else {
245.                 command = 0b00000000; //otherwise use OUT0 for channel 0
246.             }
247.
248.             i2c_start(0b01011000); //send the start signal and set address
249.
250.             i2c_write(command); //write command to DAC
251.             i2c_write(dac); //send DAC voltage value
252.             i2c_stop(); //send stop signal
253.
254.     }
255.
256.     void promptUser(void){ //prompts the user with commands they can use
257.         USART_Print("\n");
258.         USART_Print("Please enter one of the following commands:\n");
259.         USART_Print("G - to receive a single voltage measurement\n");
260.         USART_Print("M,n,dt - to receive multiple voltage measurements\n");
261.         USART_Print("S,c,v - to set the DAC output voltage\n");
262.     }
263.
264.
265.     int main(void){
266.         USART_Init(MYUBRR); //initializes USART
267.         ADC_Init(); //initialize ADC
268.         i2c_init(); //initialize DAC
269.
270.         unsigned char c; //stores user input
271.         promptUser();
272.         while(1){
273.             c = USART_Recieve(); //get character from user input
274.             if(c == 'G'){
275.                 getVoltage(); //if they inputted G get single voltage
measurement
276.                 USART_Transmit('\n');
277.             }
278.             else if(c == 'M'){
279.                 readInput_M(); //if input is M read the following n and dt
values
280.                 USART_Transmit('\n');
281.             }
282.             else if(c == 'S'){
283.                 readInput_S(); //if input is S read c and v values
284.                 USART_Transmit('\n');

```

```
285.         }else{
286.             USART_Transmit(c); //otherwise print input back to user
287.             USART_Transmit('\n');
288.         }
289.     }
290. }
291. }
```

Appendix B: References

Peter Fleurys I2C Master Library

http://www.peterfleury.epizy.com/doxygen/avr-gcc-libraries/group_pfleury_ic2master.html?i=1

ATmega328P Datasheet

https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf